Team Note of ThreeMinuteSearch

gokim06, auaahks, wnstjd9288

ICPC 2026 Seoul regional

Contents

1 정렬 / 이분 탐색 관련 **1**
- 1.1 이분탐색 / 매개변수탐색 . . . 1
- 1.2 정렬 비교 함수 구현 . . . 2
- 1.3 좌표 압축 . . . 2

2 그래프/트리 (Easy) **2**
- 2.1 위상 정렬 . . . 2
- 2.2 최단 거리 - Floyd Warshall . . . 2
- 2.3 최단 거리 - Dijkstra . . . 2
- 2.4 최단 거리 - Bellman Ford . . . 3
- 2.5 유니온 파인드 + 최소 신장 트리(Kruskal) . . . 3

3 자료구조 **3**
- 3.1 세그먼트 트리 . . . 3
- 3.2 세그먼트 트리 + 레이지 프로퍼게이션 . . . 3
- 3.3 Fenwick Tree . . . 3
- 3.4 Policy Based Data Structures - Ordered Set . . . 4
- 3.5 퍼시스턴트 세그먼트 트리 . . . 4

4 그래프/트리 (Hard) **4**
- 4.1 SCC - Kosaraju . . . 4
- 4.2 BCC - Tarjan . . . 5
- 4.3 최대 유량 - Dinic . . . 5
- 4.4 MCMF . . . 6
- 4.5 이분 매칭 - Hopcroft Karp . . . 6
- 4.6 Heavy Light Decomposition . . . 7
- 4.7 Centroid Decomposition . . . 8

5 수학 **8**
- 5.1 선형 시간 체, 곱셈적 함수 전처리 . . . 8
- 5.2 확장 유클리드 알고리즘 . . . 9
- 5.3 중국인의 나머지 정리 . . . 9
- 5.4 이항 계수를 소수로 나눈 나머지 . . . 9
- 5.5 빠른 소수 판별, 소인수분해 - Miller Rabin, Pollard Rho . . . 9
- 5.6 가우스 소거법 - RREF, 랭크, 행렬식, 역행렬 . . . 10
- 5.7 다항식 곱셈(FFT) . . . 10

6 문자열 **11**
- 6.1 문자열 해싱 . . . 11
- 6.2 문자열 매칭 - KMP . . . 11
- 6.3 가장 긴 팰린드롬 부분 문자열 - Manacher . . . 11
- 6.4 문자열 매칭 - Z . . . 11
- 6.5 접미사 배열 . . . 12

7 계산 기하 **12**
- 7.1 2차원 계산 기하 템플릿 + CCW . . . 12
- 7.2 360도 각도 정렬 . . . 12
- 7.3 다각형 넓이 . . . 12
- 7.4 선분 교차 판정 . . . 12
- 7.5 다각형 내부 판별 . . . 13
- 7.6 볼록 껍질 - Graham Scan . . . 13
- 7.7 가장 먼 두 점 - Rotating Calipers . . . 13
- 7.8 볼록 다각형 내부 판별 . . . 13

8 기타 **13**
- 8.1 가장 긴 증가하는 부분 수열(LIS) . . . 13
- 8.2 삼분 탐색 . . . 13
- 8.3 C++ 랜덤, GCC 확장, 비트마스킹 트릭 . . . 14
- 8.4 빠른 입력(Fast Input from STDIN) . . . 14

9 Other Stuffs **14**
- 9.1 Sum Over Subsets Dynamic Programming . . . 14
- 9.2 Edmond's Algorithm (Directed-MST) . . . 14
- 9.3 LCA - ETT + RMQ . . . 15
- 9.4 LCA - Binary Lifting . . . 15
- 9.5 Eulerian Path/Circuit . . . 15
- 9.6 Maximum Subarray Segment Tree . . . 15
- 9.7 Offline Queries - Mo's Algorithm . . . 16
- 9.8 2-Dimensional Segment Tree - Recursive . . . 16
- 9.9 2-Dimensional Segment Tree - Iterative . . . 17
- 9.10 2-Dimensional Fenwick Tree . . . 17

10 template **17**
- 10.1 cpp template . . . 17
- 10.2 Makefile . . . 18

11 문제 풀이 체크리스트 **18**

12 empty **19**

1 정렬 / 이분 탐색 관련

1.1 이분탐색 / 매개변수탐색

Time Complexity: $O(\log N)$

```cpp
int lowerBound(const vector<int>& arr, int target) {
    int s = 0;
    int e = arr.size();

    while (s < e) {
        int mid = (s + e) / 2;
        if (arr[mid] >= target) {
            e = mid;
        } else {
            s = mid + 1;
        }
    }
    return s;
}

// target 초과(>)가 처음 나타나는 인덱스 반환
int upperBound(const vector<int>& arr, int target) {
    int s = 0;
    int e = arr.size();

    while (s < e) {
        int mid = (s + e) / 2;
        if (arr[mid] > target) {
```

```
            e = mid;
        } else {
            s = mid + 1;
        }
    }
    return s;
}

int parametricSearchMax(const vector<int>& arr, int target) {
    int s = 0;
    int e = arr.size();
    int ans = 0;

    while (s <= e) {
        int mid = (s + e) / 2;

        if (check(mid, arr, target)) {
            ans = mid;
            s = mid + 1;
        } else {
            e = mid - 1;
        }
    }
    return ans;
}
```

1.2 정렬 비교 함수 구현

Usage: $PointlpX\$\backslash)\imath Compareh_{\lrcorner}\backslash^{\smile}, 0\Gamma^{\smile}$
Time Complexity: –

```
struct Point{
int x, y;
// 방법1: 연산자오버로딩
// x좌표오름차순, x좌표같으면y좌표오름차순
// sort(시작주소, 끝주소)
bool operator < (const Point &p) const {
if(x != p.x) return x < p.x;
else return y < p.y;
}
};
// 방법2: 비교함수구현
// sort(시작주소, 끝주소, Compare)
bool Compare(const Point &a, const Point &b){
if(a.x != b.x) return a.x < b.x;
else return a.y < b.y;
}
vector<Point> V;
V.push_back({1, 2});
V.push_back({1, 1});
V.push_back({2, 3});
sort(V.begin(), V.end()); // 방법1
sort(V.begin(), V.end(), Compare); // 방법2
```

1.3 좌표 압축

Usage: $X\Gamma\Gamma|Xt\emptyset[0,N)\Xi X_{\lrcorner}\backslash U$
Time Complexity: $O(N \log N)$

```
// Time Complexity: O(N log N)
// 원소의대소관계를유지하면서[0, N) 범위의수로압축함
// ex. {50, 31, 24, 10, 46, 10} -> {4, 2, 1, 0, 3, 0}
int N = 6, A[6] = {50, 31, 24, 10, 46, 10};
vector<int> C;
for(int i=0; i<N; i++) C.push_back(A[i]);
sort(C.begin(), C.end());
C.erase(unique(C.begin(), C.end()), C.end());
for(int i=0; i<N; i++){
A[i] = lower_bound(C.begin(), C.end(), A[i]) - C.begin();
}
for(int i=0; i<N; i++) cout << A[i] << " "; // 4 2 1 0 3 0
```

2 그래프/트리 (Easy)

2.1 위상 정렬

Usage: $AddEdge\backslash\Xi\Gamma TopSort8$
Time Complexity: $O(V+E)$

```
// Time Complexity: O(V + E)
int N, M, In[1010];
vector<int> G[1010];
void AddEdge(int s, int e){ G[s].push_back(e); In[e]++; }
void TopSort(){
queue<int> Q;
for(int i=1; i<=N; i++) if(!In[i]) Q.push(i);
while(!Q.empty()){
int v = Q.front(); Q.pop();
cout << v << " ";
for(auto i : G[v]) if(!--In[i]) Q.push(i);
}
}
```

2.2 최단 거리 - Floyd Warshall

Usage: $Init < \backslash\Phi 0T, AddEdge\backslash\Xi\Gamma Run8$
Time Complexity: $O(V^3)$

```
// Time Complexity: O(V 3)
int N, G[111][111];
void Init(){ // 시작전에호출해야함
memset(G, 0x3f, sizeof G);
for(int i=1; i<=N; i++) G[i][i] = 0;
}
// s에서e로가는가중치w 간선추가
void AddEdge(int s, int e, int w){
G[s][e] = min(G[s][e], w);
}
void Run(){
for(int k=1; k<=N; k++)
for(int i=1; i<=N; i++)
for(int j=1; j<=N; j++)
G[i][j] = min(G[i][j], G[i][k] + G[k][j]);
}
```

2.3 최단 거리 - Dijkstra

Usage: $L_{\lrcorner}\Gamma\jmath X\Gamma\Xi\emptyset\imath s@)\imath t|x\backslash Dijkstra8$
Time Complexity: $O(E \log E)$

```
// Time Complexity: O(E log E)
ll D[505050], P[505050];
vector<pair<ll,ll>> G[505050]; // {정점, 가중치}
// 주의: 가중치>= 0, 음수있으면bellman ford 사용
// s -> t 최단경로출력
void Dijkstra(int s, int t){
memset(D, 0x3f, sizeof D);
priority_queue<pair<ll,ll>, vector<pair<ll,ll>>, greater<>> Q;
Q.emplace(D[s]=0, s);
while(!Q.empty()){
auto [c,v] = Q.top(); Q.pop();
if(c == D[v]) for(auto [i,w] : G[v]) if(D[i] > c + w) Q.emplace(D[i]=c+w, i), P[i] = v;
```

```
}
vector<int> path;
for(int i=t; i!=s; i=P[i]) path.push_back(i);
path.push_back(s);
reverse(path.begin(), path.end());
for(auto i : path) cout << i << " ";
}
```

2.4 최단 거리 - Bellman Ford

Usage: $AddEdge\backslash\Xi\Gamma\imath s|x\backslash Run8$

Time Complexity: $O(VE)$

```
// Time Complexity: O(V E)
int N, M; ll D[555]; // 주의: 웬만하면long long으로잡는게좋음
vector<tuple<int,int,ll>> E; // {from, to, weight}
void AddEdge(int s, int e, int w){
E.emplace_back(s, e, w);
}
// s에서도달가능한음수사이클있으면false 반환
bool Run(int s){
memset(D, 0x3f, sizeof D);
ll INF = D[0];
D[s] = 0;
for(int iter=1; iter<=N; iter++){
bool changed = false;
for(auto [u,v,w] : E){
if(D[u] == INF) continue;
if(D[v] > D[u] + w) D[v] = D[u] + w, changed = true;
}
if(iter == N && changed) return false;
}
return true;
}
```

2.5 유니온 파인드 + 최소 신장 트리(Kruskal)

Usage: $Find@Merge\backslash(\text{fix}\ ,\Xi D\check{}\ ,\backslash Kruskal\ ,$

Time Complexity: $UF: O(\log N), MST: O(E \log E)$

```
// Time Complexity: UF: 연산마다O(logN), MST: O(E log E)
int N, M, P[10101];
int Find(int v){ return v == P[v] ? v : P[v] = Find(P[v]); }
bool Merge(int u, int v){
u = Find(u); v = Find(v);
if(u == v) return false;
P[u] = v; return true;
}
int main(){
cin >> N >> M;
vector<tuple<int,int,int>> E; // {weight, from, to}
for(int i=1,u,v,w; i<=M; i++){
cin >> u >> v >> w;
E.emplace_back(w, u, v);
}
sort(E.begin(), E.end());
for(int i=1; i<=N; i++) P[i] = i;
long long res = 0;
for(auto [w,u,v] : E) if(Merge(u, v)) res += w;
cout << res;
}
```

3 자료구조

3.1 세그먼트 트리

Usage: $Set<\backslash|XSum<\backslash[l,r]l\Xi Xi$

Time Complexity: $O(\log N)$

```
// Time Complexity: O(log N)
// SZ: N보다크거나같은2^k 꼴의수
// 13만-> 1 << 17 (131072), 26만-> 1 << 18 (262144)
// 52만-> 1 << 19 (524288), 100만-> 1 << 20 (1048576)
constexpr int SZ = 1 << 20;
ll T[SZ<<1];
void Set(int x, ll v){ // x번째수를v로지정, x는0 이상SZ 미만
x += SZ; T[x] = v;
while(x /= 2) T[x] = T[x*2] + T[x*2+1];
}
ll Sum(int l, int r){ // [l, r] 구간의합
ll res = 0;
for(l+=SZ, r+=SZ; l<=r; l/=2, r/=2){
if(l % 2 == 1) res += T[l++];
if(r % 2 == 0) res += T[r--];
}
return res;
}
```

3.2 세그먼트 트리 + 레이지 프로퍼게이션

Usage: $RangeAdd\backslash l\Xi\grave{}\ DTXRangeSum<\backslash l\Xi i$

Time Complexity: $O(\log N)$

```
// Time Complexity: O(log N)
// SZ: N보다크거나같은2^k 꼴의수
// 13만-> 1 << 17 (131072), 26만-> 1 << 18 (262144)
// 52만-> 1 << 19 (524288), 100만-> 1 << 20 (1048576)
constexpr int SZ = 1 << 20;
ll T[SZ<<1], L[SZ<<1];
void Push(int node, int s, int e){
if(L[node] == 0) return;
T[node] += (e - s + 1) * L[node];
if(s != e) L[node*2] += L[node], L[node*2+1] += L[node];
L[node] = 0;
}
// [l, r]번째수에v를더함, 0 <= l <= r < SZ
void RangeAdd(int l, int r, ll v, int node=1, int s=0, int e=SZ-1){
Push(node, s, e);
if(r < s || e < l) return;
if(l <= s && e <= r){ L[node] += v; Push(node, s, e); return; }
int m = (s + e) / 2;
RangeAdd(l, r, v, node*2, s, m);
RangeAdd(l, r, v, node*2+1, m+1, e);
T[node] = T[node*2] + T[node*2+1];
}
// [l, r]번째수의합을구함
ll RangeSum(int l, int r, int node=1, int s=0, int e=SZ-1){
Push(node, s, e);
if(r < s || e < l) return 0;
if(l <= s && e <= r) return T[node];
int m = (s + e) / 2;
return RangeSum(l, r, node*2, s, m) + RangeSum(l, r, node*2+1, m+1, e);
}
```

3.3 Fenwick Tree

Usage: $range_update\backslash l\Xi\grave{}\ DTXrsq\backslash l\Xi i$

Time Complexity: $O(\log N)$

```
// Time Complexity: O(log N)
// author: https://github.com/ncduy0303
int n, q;
ll ft1[MAX_N], ft2[MAX_N];
void update(ll *ft, int x, ll v) {
for (; x <= n; x += LSOne(x))
ft[x] += v;
}
void range_update(int l, int r, ll v) {
update(ft1, l, v);
update(ft1, r + 1, -v);
update(ft2, l, v * (l - 1));
update(ft2, r + 1, -v * r);
}
ll sum(ll *ft, int x) {
ll res = 0;
for (; x; x -= LSOne(x))
res += ft[x];
return res;
}
// sum of all elements in [1, x]
ll ps(int x) {
return sum(ft1, x) * x - sum(ft2, x);
}
// range sum query of [l, r]
ll rsq(int l, int r) {
return ps(r) - ps(l - 1);
}
```

3.4 Policy Based Data Structures - Ordered Set

Usage: $find_by_order\backslash k, order_of_key\backslash \check{}@\o slash$

Time Complexity: $O(\log N)$

```
// Time Complexity: O(log N)
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
template <typename T>
using ordered_set = tree<
// T,
// null_type,
less<T>,
// rb_tree_tag,
// tree_order_statistics_node_update
>;
int main() {
ios::sync_with_stdio(false);
cin.tie(nullptr);
ordered_set<int> s;
s.insert(10);
s.insert(20);
s.insert(30);
s.insert(40);
// find_by_order(k): returns iterator to k-th element (0-indexed)
cout << *s.find_by_order(0) << "\n"; // 10
cout << *s.find_by_order(2) << "\n"; // 30
// order_of_key(x): returns number of elements strictly smaller than x
cout << s.order_of_key(25) << "\n"; // 2 (10, 20 < 25)
cout << s.order_of_key(40) << "\n"; // 3
// You can use it to find the nth element online in O(log N)
int n = 2;
cout << "The " << n << "-th smallest element is " << *s.find_by_order(n - 1) << "\n";
return 0;
}
```

3.5 퍼시스턴트 세그먼트 트리

Usage: $\Xi init(root[0], s, e)8$

Time Complexity: $O(\log N)$

```
// Usage: call init(root[0], s, e) before use
struct PSTNode{
PSTNode *l, *r; int v;
PSTNode(){ l = r = nullptr; v = 0; }
};
PSTNode *root[101010];
PST(){ memset(root, 0, sizeof root); } // constructor
void init(PSTNode *node, int s, int e){
if(s == e) return;
int m = s + e >> 1;
node->l = new PSTNode; node->r = new PSTNode;
init(node->l, s, m); init(node->r, m+1, e);
}
void update(PSTNode *prv, PSTNode *now, int s, int e, int x){
if(s == e){ now->v = prv ? prv->v + 1 : 1; return; }
int m = s + e >> 1;
if(x <= m){
now->l = new PSTNode; now->r = prv->r;
update(prv->l, now->l, s, m, x);
}
else{
now->r = new PSTNode; now->l = prv->l;
update(prv->r, now->r, m+1, e, x);
}
int t1 = now->l ? now->l->v : 0;
int t2 = now->r ? now->r->v : 0;
now->v = t1 + t2;
}
int kth(PSTNode *prv, PSTNode *now, int s, int e, int k){
if(s == e) return s;
int m = s + e >> 1, diff = now->l->v - prv->l->v;
if(k <= diff) return kth(prv->l, now->l, s, m, k);
else return kth(prv->r, now->r, m+1, e, k-diff);
}
```

4 그래프/트리 (Hard)

4.1 SCC - Kosaraju

Usage: $\check{})DFS\backslash \o slash \o slash |\Gamma\backslash)DFS\backslash SCCl1$

Time Complexity: $O(V + E)$

```
// Time Complexity: O(V + E)
constexpr int SZ = 101010;
int N, M;
vector<int> G[SZ], R[SZ]; // G: 원래 그래프, R: 역방향 그래프
vector<int> V; // DFS1 종료 순서
vector<vector<int>> S; // SCC별 정점 리스트
int C[SZ];
// C[i]: i번정점이속한SCC 번호(0:미방문, -1:DFS1방문, 1이상:SCC ID)
// 1. 간선추가(양방향저장)
void AddEdge(int s, int e){
G[s].push_back(e); R[e].push_back(s); }
// 2. 첫번째DFS: 종료순서대로V에담기(위상정렬순서파악)
void DFS1(int v){
C[v] = -1; // 방문표시(임시)
```

```
for(auto i : G[v]) if(!C[i]) DFS1(i);
V.push_back(v); // 끝나는순서대로넣음
}
// 3. 두번째DFS: 역방향그래프에서묶기(SCC 형성)
void DFS2(int v, int c){
C[v] = c; // SCC 번호부여
S.back().push_back(v); // 현재SCC 그룹에추가
for(auto i : R[v]) if(C[i] == -1) DFS2(i, c);
}
int GetSCC(){
// 초기화(여러테스트케이스일경우필요)
// S.clear(); V.clear(); memset(C, 0, sizeof(C));
// Step 1: 정방향DFS로스택(V) 채우기
for(int i=1; i<=N; i++) if(!C[i]) DFS1(i);
// Step 2: 스택을뒤집어서역방향DFS 수행
reverse(V.begin(), V.end());
int cnt = 0;
for(auto i : V) {
if(C[i] == -1) { // DFS1에서방문했지만아직SCC 그룹이없는경우
S.emplace_back(); // 새로운SCC 그룹생성
DFS2(i, ++cnt);
// SCC 번호는1번부터시작
}
} return cnt; // 총SCC 개수반환}
```

4.2 BCC - Tarjan

Usage: $\Xi\Gamma xD_{,}X0\Xi tarjan8$

Time Complexity: $O(V+E)$

```
// Time Complexity: O(V + E)
// 1-based, 다른거호출하기전에tarjan 먼저호출해야함
vector<int> G[MAX_V]; int In[MAX_V], Low[MAX_V], P[MAX_V];
void addEdge(int s, int e){ G[s].push_back(e); G[e].push_back(s); }
void tarjan(int n){ /// Pre-Process
int pv = 0;
function<void(int,int)> dfs = [&pv,&dfs](int v, int b){
In[v] = Low[v] = ++pv; P[v] = b;
for(auto i : G[v]){
if(i == b) continue;
if(!In[i]) dfs(i, v), Low[v] = min(Low[v], Low[i]); else Low[v] = min(Low[v], In[i]);
}
};
for(int i=1; i<=n; i++) if(!In[i]) dfs(i, -1);
}
vector<int> cutVertex(int n){
vector<int> res; array<char,MAX_V> isCut; isCut.fill(0);
function<void(int)> dfs = [&dfs,&isCut](int v){
int ch = 0;
for(auto i : G[v]){
if(P[i] != v) continue; dfs(i); ch++;
if(P[v] == -1 && ch > 1) isCut[v] = 1; else if(P[v] != -1 && Low[i] >= In[v])
isCut[v]=1;
}
};
for(int i=1; i<=n; i++) if(P[i] == -1) dfs(i);
for(int i=1; i<=n; i++) if(isCut[i]) res.push_back(i);
return move(res);
}
vector<PII> cutEdge(int n){
vector<PII> res;
function<void(int)> dfs = [&dfs,&res](int v){
for(int t=0; t<G[v].size(); t++){
int i = G[v][t]; if(t != 0 && G[v][t-1] == G[v][t]) continue;
if(P[i] != v) continue; dfs(i);
if((t+1 == G[v].size() || i != G[v][t+1]) && Low[i] > In[v])
res.emplace_back(min(v,i), max(v,i));
}
};
for(int i=1; i<=n; i++) sort(G[i].begin(), G[i].end()); // multi edge -> sort
for(int i=1; i<=n; i++) if(P[i] == -1) dfs(i);
return move(res); // sort(all(res));
}
vector<int> BCC[MAX_V]; // BCC[v] = components which contains v
void vertexDisjointBCC(int n){ // allow multi edge, not allow self loop
int cnt = 0; array<char,MAX_V> vis; vis.fill(0);
function<void(int,int)> dfs = [&dfs,&vis,&cnt](int v, int c){
vis[v] = 1; if(c > 0) BCC[v].push_back(c);
for(auto i : G[v]){
if(vis[i]) continue;
if(In[v] <= Low[i]) BCC[v].push_back(++cnt), dfs(i, cnt); else dfs(i, c);
}
};
for(int i=1; i<=n; i++) if(!vis[i]) dfs(i, 0);
for(int i=1; i<=n; i++) if(BCC[i].empty()) BCC[i].push_back(++cnt);
}
```

4.3 최대 유량 - Dinic

Usage: $maximum_{f}low(s,t)\backslash s\o t\backslash X\backslash\Gamma$

Time Complexity: $O(V^2E), \Xi Xt1ttO(\min(V^{2/3}, E^{1/2})E)$

```
// Time Complexity: O(V 2E), 모든간선의용량이1이면O(min(V 2/3, E1/2)E)
// 디닉알고리즘(Dinic) 최대유량, O(E * sqrt(V)) ~ O(V^2 E)
// 사용법)
//
// Dinic<int> mf(n);
//
// mf.add_edge(u, v, cap); // 유향, 무향은왕복용량추가
//
// auto flow = mf.maximum_flow(src, sink);
//
// 최소컷은minimum_cut 호출(유량재활용)
// Directed Graph이면add_edge(s, e, c)
// Undirected Graph이면add_edge(s, e, c, c)
template<typename flow_t, flow_t MAX_U=(1<<30)>
struct Dinic{ // 0-based
struct edge_t{ int v, r; flow_t c, f; };
int n;
vector<vector<edge_t>> g;
vector<int> lv, idx;
Dinic(int n) : n(n) { clear(); }
void clear(){
g = vector<vector<edge_t>>(n);
lv = vector<int>(n, 0);
idx = vector<int>(n, 0);
}
// add_edge: s→e 용량c1, e→s 역간선용량c2 (기본0)
void add_edge(int s, int e, flow_t c1, flow_t c2=flow_t(0)){
g[s].push_back({e, (int)g[e].size(), c1, 0});
g[e].push_back({s, (int)g[s].size()-1, c2, 0});
}
// bfs: 레벨그래프구성(limit 사용하면Scaling Dinic)
bool bfs(int s, int t, flow_t limit=1){
fill(lv.begin(), lv.end(), 0);
queue<int> que; que.push(s); lv[s] = 1;
while(!que.empty()){
```

```
int v = que.front(); que.pop();
for(const auto &e : g[v]) if(!lv[e.v] && e.c - e.f >= limit) que.push(e.v), lv[e.v] =
lv[v] + 1;
}
return lv[t] != 0;
}
// dfs: Blocking flow 탐색
flow_t dfs(int v, int t, flow_t fl=MAX_U){
if(v == t || fl == flow_t(0)) return fl;
for(int &i=idx[v]; i<g[v].size(); i++){
auto &e = g[v][i];
if(lv[e.v] != lv[v] + 1 || e.c - e.f == flow_t(0)) continue;
flow_t now = dfs(e.v, t, min(fl, e.c - e.f));
if(now == flow_t(0)) continue;
e.f += now; g[e.v][e.r].f -= now;
return now;
}
return 0;
}
// maximum_flow: s→t 최대유량반환
flow_t maximum_flow(int s, int t){
flow_t flow = 0, augment = 0;
while(bfs(s, t)){
fill(idx.begin(), idx.end(), 0);
while((augment=dfs(s, t)) != flow_t(0)) flow += augment;
}
return flow;
}
// {최소컷비용, s와같은집합, t와같은집합, 절단간선}
// minimum_cut: (최소컷값, S집합, T집합, 절단간선) 반환
tuple<flow_t, vector<int>, vector<int>, vector<pair<int,int>>> minimum_cut(int s, int t){
flow_t flow = maximum_flow(s, t);
vector<int> a, b;
vector<pair<int,int>> edges;
bfs(s, t, 1);
for(int i=0; i<n; i++) (lv[i] ? a : b).push_back(i);
for(auto i : a) for(auto e : g[i]) if(e.c != flow_t(0) && !lv[e.v])
edges.emplace_back(i, e.v);
return {flow, a, b, edges};
}
};
```

4.4 MCMF

Usage: –
Time Complexity: –

```
// 최소비용유량(Min-Cost Max-Flow), O(Flow * E)
// 사용법)
//
// MinCostFlow<int,long long> mcf(n);
//
// mcf.add_edge(u, v, cap, cost);
//
// auto [flow, cost] = mcf.run(src, sink);
// 가능한만큼흘림
//
// auto [flow, cost] = mcf.run(src, sink, need);
// need 만큼만흘림
template<typename flow_t=int, typename cost_t=long long, flow_t MAX_U=(1<<30), cost_t
MAX_C=(1LL<<60)>
struct MinCostFlow{ // 0-based
struct edge_t{ int v, r; flow_t c; cost_t d; };
int n;
vector<vector<edge_t>> g;
vector<int> prv, idx, chk;
vector<cost_t> dst;
MinCostFlow(int n) : n(n) { clear(); }
void clear(){
g = vector<vector<edge_t>>(n);
prv = idx = chk = vector<int>(n);
dst = vector<cost_t>(n);
}
// add_edge: 용량c, 비용d (역간선자동추가)
void add_edge(int s, int e, flow_t c, cost_t d){
g[s].push_back({e, (int)g[e].size(), c, d});
g[e].push_back({s, (int)g[s].size()-1, 0, -d});
}
// SPFA 기반최단경로탐색(음수비용허용)
bool find_path(int s, int t){
fill(chk.begin(), chk.end(), 0);
fill(dst.begin(), dst.end(), MAX_C);
queue<int> que; que.push(s); dst[s] = 0; chk[s] = 1;
while(!que.empty()){
int v = que.front(); que.pop(); chk[v] = 0;
for(int i=0; i<g[v].size(); i++){
const auto &e = g[v][i];
if(e.c > 0 && dst[e.v] > dst[v] + e.d){
dst[e.v] = dst[v] + e.d; prv[e.v] = v; idx[e.v] = i;
if(!chk[e.v]) que.push(e.v), chk[e.v] = 1;
}
}
}
return dst[t] < MAX_C;
}
// augment: 단일경로로가능한만큼혹은k 제한만큼유량흘리기
pair<flow_t, cost_t> augment(int s, int t, flow_t k=-1){
if(!find_path(s, t)) return {0, 0};
flow_t fl = MAX_U;
for(int i=t; i!=s; i=prv[i]) fl = min(fl, g[prv[i]][idx[i]].c);
if(k != -1) fl = min(fl, k);
for(int i=t; i!=s; i=prv[i]){
g[prv[i]][idx[i]].c -= fl;
g[i][g[prv[i]][idx[i]].r].c += fl;
}
return {fl, fl * dst[t]};
}
// run: 전체루프돌며누적유량/비용반환
pair<flow_t, cost_t> run(int s, int t, flow_t k=-1){
flow_t flow = 0; cost_t cost = 0;
while(true){
auto [fl,cst] = augment(s, t, k);
if(fl == 0) break;
flow += fl; cost += cst;
if(k != -1) k -= fl;
}
return {flow, cost};
}
};
```

4.5 이분 매칭 - Hopcroft Karp

Usage: –
Time Complexity: $O(E\sqrt{V})$

```
// Time Complexity: O(E
// √
V )
```

```cpp
// 홉크로프트-카프(Hopcroft-Karp) 이분매칭, O(E sqrt(V))
// n: 왼쪽정점수, m: 오른쪽정점수, 0-based 인덱스
// 사용방법)
//
HopcroftKarp hk(n, m);
//
hk.add_edge(u, v); // 왼쪽u, 오른쪽v 연결
//
int match = hk.maximum_matching();
//
auto edges = hk.maximum_matching_edges(); // 매칭간선목록
//
hk.minimum_vertex_cover(); // 최소정점커버계산
struct HopcroftKarp{
int n, m;
vector<vector<int>> g;
vector<int> dst, le, ri;
vector<char> visit, track;
HopcroftKarp(int n, int m) : n(n), m(m) { clear(); }
void clear(){
g = vector<vector<int>>(n); dst = vector<int>(n, 0);
le = vector<int>(n, -1); ri = vector<int>(m, -1);
visit = vector<char>(n, 0); track = vector<char>(n+m, 0);
}
void add_edge(int s, int e){ g[s].push_back(e); }
bool bfs(){
bool res = false;
queue<int> que;
fill(dst.begin(), dst.end(), 0);
for(int i=0; i<n; i++) if(le[i] == -1) que.push(i), dst[i] = 1;
while(!que.empty()){
int v = que.front(); que.pop();
for(auto i : g[v]){
if(ri[i] == -1) res = true;
else if(!dst[ri[i]]) dst[ri[i]] = dst[v] + 1, que.push(ri[i]);
}
}
return res;
}
bool dfs(int v){
if(visit[v]) return false;
visit[v] = 1;
for(auto i : g[v]){
if(ri[i] == -1 || !visit[ri[i]] && dst[ri[i]] == dst[v] + 1 && dfs(ri[i])){
le[v] = i; ri[i] = v; return true;
}
}
return false;
}
int maximum_matching(){
int res = 0;
fill(le.begin(), le.end(), -1);
fill(ri.begin(), ri.end(), -1);
while(bfs()){
fill(visit.begin(), visit.end(), 0);
for(int i=0; i<n; i++) if(le[i] == -1) res += dfs(i);
}
return res;
}
vector<pair<int,int>> maximum_matching_edges(){
int matching = maximum_matching();
vector<pair<int,int>> edges; edges.reserve(matching);
for(int i=0; i<n; i++) if(le[i] != -1) edges.emplace_back(i, le[i]);
return edges;
}
void dfs_track(int v){
if(track[v]) return; track[v] = 1;
for(auto i : g[v]) track[n+i] = 1, dfs_track(ri[i]);
}
tuple<vector<int>, vector<int>, int> minimum_vertex_cover(){
int matching = maximum_matching();
fill(track.begin(), track.end(), 0);
for(int i=0; i<n; i++) if(le[i] == -1) dfs_track(i);
vector<int> lv, rv;
for(int i=0; i<n; i++) if(!track[i]) lv.push_back(i);
for(int i=0; i<m; i++) if(track[n+i]) rv.push_back(i);
assert(lv.size() + rv.size() == matching);
return {lv, rv, lv.size() + rv.size()};
}
};
```

4.6 Heavy Light Decomposition

Usage: ˇıΓȷX1ΞΓȷXΓ\Ξ\¸

Time Complexity: Ξ$O(N)$, $O(T(N)\log N)$

```cpp
// Time Complexity: 전처리O(N), 쿼리O(T(N) log N)
// sz:서브트리크기, dep:깊이, par:부모, top:체인헤드, in:DFS방문순서(SegTree의인덱스)
vector<int> adj[SZ]; //pv: 방문순서카운터
int sz[SZ], dep[SZ], par[SZ], top[SZ], in[SZ], pv;
// 1. DFS 1: 서브트리크기계산및Heavy Edge를0번으로보냄
void dfs_sz(int v, int p = 0) {
sz[v] = 1;
par[v] = p;
dep[v] = dep[p] + 1;
for (auto &next : adj[v]) {
if (next == p) continue;
dfs_sz(next, v);
sz[v] += sz[next];
// 가장무거운자식을adj[v][0]으로스왑(Heavy Edge)
if (sz[next] > sz[adj[v][0]] || adj[v][0] == p) {
swap(next, adj[v][0]);
}}}
// 2. DFS 2: DFS 순서(in) 부여및Top 설정
void dfs_hld(int v) {
in[v] = ++pv;
for (auto next : adj[v]) {
if (next == par[v]) continue;
// Heavy Edge(0번)면Top 유지, 아니면자신이새로운Top
top[next] = (next == adj[v][0]) ? top[v] : next;
dfs_hld(next);
}}
// 3. 경로쿼리(u, v 경로의합/최대값등)
long long query_path(int u, int v) {
long long ret = 0;
while (top[u] != top[v]) {
if (dep[top[u]] < dep[top[v]]) swap(u, v);
int st = top[u];
ret += seg_query(in[st], in[u]); // 체인구간쿼리
u = par[st];
}
if (dep[u] > dep[v]) swap(u, v);
// [Type A] 정점가중치(Vertex Weight)
ret += seg_query(in[u], in[v]);
// [Type B] 간선가중치(Edge Weight)
// LCA(u)는경로에포함되지않으므로in[u] + 1 부터시작
// if (u != v) ret += seg_query(in[u] + 1, in[v]);
```

```
return ret;
}
// 4-1. 정점업데이트(u의값을val로)
void update_vertex(int u, int val) { seg_update(in[u], val); }
// 4-2. 간선업데이트(u-v 간선값을val로)
void update_edge(int u, int v, int val) {
// 둘중깊이가더깊은쪽이자식노드이며, 간선값을들고있음
if (dep[u] < dep[v]) seg_update(in[v], val);
else seg_update(in[u], val); }
// 5. 초기화및실행예시
void init_hld(int root = 1) {
pv = 0; dfs_sz(root);
top[root] = root;
dfs_hld(root);
// 1. 정점가중치인경우:
//
for(int i=1; i<=N; i++) seg_update(in[i], Cost[i]);
// 2. 간선가중치인경우(입력이u, v, w): 트리에는일단간선만넣음
//
// dfs가다끝난뒤에수행해야dep 비교가능
//
if(dep[u] < dep[v]) seg_update(in[v], w); //자식에게가중치박아넣음
//
else seg_update(in[u], w);
}
```

4.7 Centroid Decomposition

Usage: –
Time Complexity: –

```
// Time Complexity: 전처리O(N log N), 쿼리O(log N)
// sz:서브트리크기, dep:깊이, par:부모, top:체인헤드, in:DFS방문순서(SegTree의인덱스)
vector<int> adj[SZ]; //pv: 방문순서카운터
int sz[SZ], dep[SZ], par[SZ], top[SZ], in[SZ], pv;
// 1. DFS 1: 서브트리크기계산및Heavy Edge를0번으로보냄
void dfs_sz(int v, int p = 0) {
sz[v] = 1;
par[v] = p;
dep[v] = dep[p] + 1;
for (auto &next : adj[v]) {
if (next == p) continue;
dfs_sz(next, v);
sz[v] += sz[next];
// 가장무거운자식을adj[v][0]으로스왑(Heavy Edge)
if (sz[next] > sz[adj[v][0]] || adj[v][0] == p) {
swap(next, adj[v][0]);
}}}
// 2. DFS 2: DFS 순서(in) 부여및Top 설정
void dfs_hld(int v) {
in[v] = ++pv;
for (auto next : adj[v]) {
if (next == par[v]) continue;
// Heavy Edge(0번)면Top 유지, 아니면자신이새로운Top
top[next] = (next == adj[v][0]) ? top[v] : next;
dfs_hld(next);
}}
// 3. 경로쿼리(u, v 경로의합/최대값등)
long long query_path(int u, int v) {
long long ret = 0;
while (top[u] != top[v]) {
if (dep[top[u]] < dep[top[v]]) swap(u, v);
int st = top[u];
ret += seg_query(in[st], in[u]); // 체인구간쿼리
u = par[st];
}
if (dep[u] > dep[v]) swap(u, v);
// [Type A] 정점가중치(Vertex Weight)
ret += seg_query(in[u], in[v]);
// [Type B] 간선가중치(Edge Weight)
// LCA(u)는경로에포함되지않으므로in[u] + 1 부터시작
// if (u != v) ret += seg_query(in[u] + 1, in[v]);
return ret;
}
// 4-1. 정점업데이트(u의값을val로)
void update_vertex(int u, int val) { seg_update(in[u], val); }
// 4-2. 간선업데이트(u-v 간선값을val로)
void update_edge(int u, int v, int val) {
// 둘중깊이가더깊은쪽이자식노드이며, 간선값을들고있음
if (dep[u] < dep[v]) seg_update(in[v], val);
else seg_update(in[u], val); }
// 5. 초기화및실행예시
void init_hld(int root = 1) {
pv = 0; dfs_sz(root);
top[root] = root;
dfs_hld(root);
// 1. 정점가중치인경우:
//
for(int i=1; i<=N; i++) seg_update(in[i], Cost[i]);
// 2. 간선가중치인경우(입력이u, v, w): 트리에는일단간선만넣음
//
// dfs가다끝난뒤에수행해야dep 비교가능
//
if(dep[u] < dep[v]) seg_update(in[v], w); //자식에게가중치박아넣음
//
else seg_update(in[u], w);
}
```

5 수학

5.1 선형 시간 체, 곱셈적 함수 전처리

Usage: $Sieve\backslash ¸ffHΔh¸Ξ$
Time Complexity: $ΞO(P), O(\log P)$

```
// sigma 계산하는부분제외하면O(n), sigma 계산은O(n log log n)
// pw(j, e[i*j]) 모두전처리하면O(n)에계산할수있음
// prime: n 이하소수목록
// sp : 최소소인수, 소수라면0
// tau : 약수개수, sigma : 약수합
// phi : n 이하자연수중n과서로소인개수
// mu : 2번이상곱해진소인수가있으면0, 그렇지않으면(-1)^(소인수개수)
// e[i] : i에sp[i]가곱해진횟수
vector<int> prime;
int sp[sz], e[sz], phi[sz], mu[sz], tau[sz], sigma[sz];
phi[1] = mu[1] = tau[1] = sigma[1] = 1;
for(int i=2; i<=n; i++){
if(!sp[i]){
prime.push_back(i);
e[i] = 1; phi[i] = i-1; mu[i] = -1; tau[i] = 2; sigma[i] = i+1;
}
for(auto j : prime){
if(i*j >= sz) break;
sp[i*j] = j;
if(i % j == 0){
e[i*j] = e[i]+1; phi[i*j] = phi[i]*j; mu[i*j] = 0;
tau[i*j] = tau[i]/e[i*j]*(e[i*j]+1);
sigma[i*j] = sigma[i]*(j-1)/(pw(j, e[i*j])-1)*(pw(j, e[i*j]+1)-1)/(j-1);//overflow
```

```
break;
}
e[i*j] = 1; phi[i*j] = phi[i] * phi[j]; mu[i*j] = mu[i] * mu[j];
tau[i*j] = tau[i] * tau[j]; sigma[i*j] = sigma[i] * sigma[j];
}
}
```

5.2 확장 유클리드 알고리즘

Usage: $UtL < \backslash\backslash\Gamma'' ,@ ,$

Time Complexity: $O(\log \max(a, b))$

```
// Time Complexity: O(log max(a, b))
// 정수a, b 주어지면ax + by = gcd(a, b) = g
// 를만족하는정수{g, x, y} 반환
tuple<ll,ll,ll> ext_gcd(ll a, ll b){
if(b == 0) return {a, 1, 0};
auto [g,x,y] = ext_gcd(b, a % b);
return {g, y, x - a/b * y};
}
```

5.3 중국인의 나머지 정리

Usage: $UtLDtti$

Time Complexity: $O(k \log m)$

```
// Time Complexity: O(k log m)
// a = a1 (mod m1), a = a2 (mod m2)를만족하는{a, lcm(m1, m2)} 반환
// 만약a가존재하면0 <= a < lcm(m1, m2) 에서유일하게존재
// a가존재하지않는경우{-1, -1} 반환
ll mod(ll a, ll b){ return (a %= b) >= 0 ? a : a + b; }
pair<ll,ll> crt(ll a1, ll m1, ll a2, ll m2){
ll g = gcd(m1, m2), m = m1 / g * m2;
if((a2 - a1) % g) return {-1, -1};
ll md = m2/g, s = mod((a2-a1)/g, m2/g);
ll t = mod(get<1>(ext_gcd(m1/g%md, m2/g)), md);
return { a1 + s * t % md * m1, m };
}
// a = a_i (mod m_i)를만족하는{a, lcm(m_1, ... , m_k)} 반환
// a가존재하지않는경우{-1, -1} 반환
pair<ll,ll> crt(const vector<ll> &a, const vector<ll> &m){
ll ra = a[0], rm = m[0];
for(int i=1; i<m.size(); i++){
auto [aa,mm] = crt(ra, rm, a[i], m[i]);
if(mm == -1) return {-1, -1}; else tie(ra,rm) = tie(aa,mm);
}
return {ra, rm};
}
```

5.4 이항 계수를 소수로 나눈 나머지

Usage: –

Time Complexity: –

```
// Time Complexity: 전처리O(P), 쿼리O(log P)
// Lucas C(13);
// C.calc(5, 3) = 5C3 = 10
// C.calc(10, 2) = 10C2 % 13 = 45 % 13 = 6
// 주의: P는소수
// P가크고(약10억) n,r이작으면(1000만이하)
// 생성자에서fac, inv를1000만까지만구해도됨
struct Lucas{ // init : O(P), query : O(log P)
const size_t P;
vector<ll> fac, inv;
ll Pow(ll a, ll b){
ll res = 1;
for(; b; b>>=1, a=a*a%P) if(b & 1) res = res * a % P;
return res;
}
Lucas(size_t P) : P(P), fac(P), inv(P) {
fac[0] = 1; for(int i=1; i<P; i++) fac[i] = fac[i-1] * i % P;
inv[P-1] = Pow(fac[P-1], P-2); for(int i=P-2; ~i; i--) inv[i] = inv[i+1] * (i+1) %
P;
}
ll small(ll n, ll r) const { return r <= n ? fac[n] * inv[r] % P * inv[n-r] % P : 0LL;
}
ll calc(ll n, ll r) const {
if(n < r || n < 0 || r < 0) return 0;
if(!n || !r || n == r) return 1; else return small(n%P, r%P) * calc(n/P, r/P) % P;
}
};
// Lucas Theorem:
// \m
// n
// {
// ≡Qk
i=0
// \mi
// ni
// {
( mod p),
where m = mkpk + mk1pk1 + · · · + m1p + m0,
and n = nkpk + nk1pk1 + · · · + n1p + n0,
// and p is prime.
// Here,
// \m
// n
// {
= 0 if m < n.
```

5.5 빠른 소수 판별, 소인수분해 - Miller Rabin, Pollard Rho

Usage: $LSieve()8$

Time Complexity: $IsPrime : O(\log^2 N), Factorize : "O(N^{1/4})$

```
// Usage: 처음에Sieve() 호출해야함
// Time Complexity: IsPrime: O(log2 N), Factorize: 약O(N1/4)
constexpr int SZ = 10'000'000;
bool PrimeCheck[SZ+1]; vector<int> Primes;
void Sieve(){
memset(PrimeCheck, true, sizeof PrimeCheck);
PrimeCheck[0] = PrimeCheck[1] = false;
for(int i=2; i<=SZ; i++){
if(PrimeCheck[i]) Primes.push_back(i);
for(auto j : Primes){
if(i*j > SZ) break;
PrimeCheck[i*j] = false;
if(i % j == 0) break;
}
}
}
using ll = long long;
using ull = unsigned long long;
ull MulMod(ull a, ull b, ull c){ return (__uint128_t)a * b % c; }
ull PowMod(ull a, ull b, ull c){
ull res = 1; a %= c;
for(; b; b>>=1, a=MulMod(a,a,c)) if(b & 1) res = MulMod(res,a,c);
return res;
}
bool MillerRabin(ull n, ull a){
```

```
if(a % n == 0) return true;
int cnt = __builtin_ctzll(n - 1);
ull p = PowMod(a, n >> cnt, n);
if(p == 1 || p == n - 1) return true;
while(cnt--) if((p=MulMod(p,p,n)) == n - 1) return true;
return false;
}
bool IsPrime(ll n){
if(n <= SZ) return PrimeCheck[n];
if(n <= 2) return n == 2;
if(n % 2 == 0 || n % 3 == 0 || n % 5 == 0 || n % 7 == 0 || n % 11 == 0) return false;
// 32bit integer: {2, 7, 61}
for(int p : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) if(!MillerRabin(n, p))
return false;
return true;
}
ll Rho(ll n){
while(true){
ll x = rand() % (n - 2) + 2, y = x, c = rand() % (n - 1) + 1;
while(true){
x = (MulMod(x, x, n) + c) % n;
y = (MulMod(y, y, n) + c) % n;
y = (MulMod(y, y, n) + c) % n;
ll d = __gcd(abs(x - y), n);
if(d == 1) continue;
if(IsPrime(d)) return d;
else{ n = d; break; }
}
}
}
vector<pair<ll,ll>> Factorize(ll n){
vector<pair<ll,ll>> v;
int two = __builtin_ctzll(n);
if(two > 0) v.emplace_back(2, two), n >>= two;
if(n == 1) return v;
while(!IsPrime(n)){
ll d = Rho(n), cnt = 0;
while(n % d == 0) cnt++, n /= d;
v.emplace_back(d, cnt);
if(n == 1) break;
}
if(n != 1) v.emplace_back(n, 1);
return v;
}
```

5.6 가우스 소거법 - RREF, 랭크, 행렬식, 역행렬

Usage: –
Time Complexity: $O(N^3)$

```
// Time Complexity: O(N3)
// T Add(T a, T b), Sub, Mul, Div 구현해야함
// bool IsZero(T x) 구현해야함
template<typename T> // return {rref, rank, det, inv}
tuple<vector<vector<T>>, int, T, vector<vector<T>>> Gauss(vector<vector<T>> a, bool
square=true){
int n = a.size(), m = a[0].size(), rank = 0;
vector<vector<T>> out(n, vector<T>(m, 0)); T det = T(1);
for(int i=0; i<n; i++) if(square) out[i][i] = T(1);
for(int i=0; i<m; i++){
if(rank == n) break;
if(IsZero(a[rank][i])){
T mx = T(0); int idx = -1; // fucking precision error
for(int j=rank+1; j<n; j++) if(mx < abs(a[j][i])) mx = abs(a[j][i]), idx = j;
if(idx == -1 || IsZero(a[idx][i])){ det = 0; continue; }
for(int k=0; k<m; k++){
a[rank][k] = Add(a[rank][k], a[idx][k]);
if(square) out[rank][k] = Add(out[rank][k], out[idx][k]);
}
}
det = Mul(det, a[rank][i]);
T coeff = Div(T(1), a[rank][i]);
for(int j=0; j<m; j++) a[rank][j] = Mul(a[rank][j], coeff);
for(int j=0; j<m; j++) if(square) out[rank][j] = Mul(out[rank][j], coeff);
for(int j=0; j<n; j++){
if(rank == j) continue;
T t = a[j][i]; // Warning: [j][k], [rank][k]
for(int k=0; k<m; k++) a[j][k] = Sub(a[j][k], Mul(a[rank][k], t));
for(int k=0; k<m; k++) if(square) out[j][k] = Sub(out[j][k], Mul(out[rank][k], t));
}
rank++;
}
return {a, rank, det, out};
}
```

5.7 다항식 곱셈(FFT)

Usage: $multiply\backslash PmX_{\lrcorner}, multiply_mod\backslash_{\lrcorner}$
Time Complexity: $O(N \log N)$

```
// Time Complexity: O(N log N)
typedef complex<double> cpx;
void FFT(vector<cpx> &a, bool inv_fft=false){
int N = a.size(); vector<cpx> root(N/2);
for(int i=1, j=0; i<N; i++){
int bit = N / 2;
while(j >= bit) j -= bit, bit >>= 1;
if(i < (j += bit)) swap(a[i], a[j]);
}
double ang = 2 * acos(-1) / N * (inv_fft ? -1 : 1);
for(int i=0; i<N/2; i++) root[i] = cpx(cos(ang*i), sin(ang*i));
for(int i=2; i<=N; i<<=1){
int step = N / i;
for(int j=0; j<N; j+=i) for(int k=0; k<i/2; k++){
cpx u = a[j+k], v = a[j+k+i/2] * root[step * k];
a[j+k] = u+v; a[j+k+i/2] = u-v;
}
}
if(inv_fft) for(int i=0; i<N; i++) a[i] /= N;
}
// [버전1] 일반곱셈: 결과값이10^14 이하일때사용(빠름)
vector<ll> multiply(const vector<ll> &_a, const vector<ll> &_b){
vector<cpx> a(all(_a)), b(all(_b));
int N = 2; while(N < a.size() + b.size()) N <<= 1;
a.resize(N); b.resize(N); FFT(a); FFT(b);
for(int i=0; i<N; i++) a[i] *= b[i];
vector<ll> ret(N); FFT(a, 1);
for(int i=0; i<N; i++) ret[i] = llround(a[i].real());
while(ret.size() > 1 && ret.back() == 0) ret.pop_back();
return ret;
}
// [버전2] 모듈러곱셈: 결과값이아주크거나모듈러연산이필요할때사용(안전함) 상수가커서조금
느릴수있음(약3~4배)
vector<ll> multiply_mod(const vector<ll> &a, const vector<ll> &b, const ull mod){
int N = 2; while(N < a.size() + b.size()) N <<= 1;
vector<cpx> v1(N), v2(N), r1(N), r2(N);
for(int i=0; i<a.size(); i++) v1[i] = cpx(a[i] >> 15, a[i] & 32767);
for(int i=0; i<b.size(); i++) v2[i] = cpx(b[i] >> 15, b[i] & 32767);
```

```
FFT(v1); FFT(v2);
for(int i=0; i<N; i++){
int j = i ? N-i : i;
cpx ans1 = (v1[i] + conj(v1[j])) * cpx(0.5, 0);
cpx ans2 = (v1[i] - conj(v1[j])) * cpx(0, -0.5);
cpx ans3 = (v2[i] + conj(v2[j])) * cpx(0.5, 0);
cpx ans4 = (v2[i] - conj(v2[j])) * cpx(0, -0.5);
r1[i] = (ans1 * ans3) + (ans1 * ans4) * cpx(0, 1);
r2[i] = (ans2 * ans3) + (ans2 * ans4) * cpx(0, 1);
}
vector<ll> ret(N); FFT(r1, true); FFT(r2, true);
for(int i=0; i<N; i++){
ll av = llround(r1[i].real()) % mod;
ll bv = ( llround(r1[i].imag()) + llround(r2[i].real()) ) % mod;
ll cv = llround(r2[i].imag()) % mod;
ret[i] = (av << 30) + (bv << 15) + cv;
ret[i] %= mod; ret[i] += mod; ret[i] %= mod;
}
while(ret.size() > 1 && ret.back() == 0) ret.pop_back();
return ret;
}
// ================ FFT 템플릿끝=================
[사용방법가이드]
vector<ll> A,B = {1, 2, 3}; // x^0, x^1, x^2 순서다항식의계수를담는다.
vector<ll> res = multiply(A, B); // A랑B곱한계수를담는다.
vector<ll> res_mod = multiply_mod(C, D, MOD); // 곱한결과를MOD로나눈나머지를요구할때. (ull
MOD)
for(int i=0; i<res.size(); i++) cout << res[i] << " "; // i번째인덱스= x^i의계수
for(int i=0; i<raw_res.size(); i++) { // 큰수곱셈자릿수올림처리
if(raw_res[i] >= 10) {
if(i == raw_res.size() - 1) raw_res.push_back(0); // 자리늘리기
raw_res[i+1] += raw_res[i] / 10;
raw_res[i] %= 10;}}
```

6 문자열

6.1 문자열 해싱

Usage: $build\backslash 8\Xi, get < \backslash 8Xt`$
Time Complexity: $build : O(N), get : O(1)$

```
// Time Complexity: build: O(N), get: O(1)
// 전처리O(N), 부분문자열의해시값을O(1)에구함
// Hashing<917, 998244353> H;
// H.build("ABCDABCD");
// assert(H.get(1, 4) == H.get(5, 8));
// 주의: get 함수의인자는1-based 닫힌구간
// 주의: M은10억근처의소수, P는M과서로소
// 1e5+3, 1e5+13, 131'071, 524'287, 1'299'709, 1'301'021
// 1e9-63, 1e9+7, 1e9+9, 1e9+103
template<long long P, long long M> struct Hashing {
vector<long long> h, p;
void build(const string &s){
int n = s.size();
h = p = vector<long long>(n+1); p[0] = 1;
for(int i=1; i<=n; i++) h[i] = (h[i-1] * P + s[i-1]) % M;
for(int i=1; i<=n; i++) p[i] = p[i-1] * P % M;
}
long long get(int s, int e) const {
long long res = (h[e] - h[s-1] * p[e-s+1]) % M;
return res >= 0 ? res : res + M;
}
};
```

6.2 문자열 매칭 - KMP

Usage: $KMP(s, p)\backslash 8s\o(4p\Gamma X\Xi X\lrcorner X$
Time Complexity: $GetFail : O(|P|), KMP : O(|S| + |P|)$

```
// Time Complexity: GetFail: O(|P|), O(|S| + |P|)
// s에서p가등장하는위치반환
// KMP("ABABCAB", "AB") = {0, 2, 5}
// KMP("AAAA", "AA") = {0, 1, 2}
vector<int> GetFail(const string &p){
int n = p.size();
vector<int> fail(n);
for(int i=1, j=0; i<n; i++){
while(j && p[i] != p[j]) j = fail[j-1];
if(p[i] == p[j]) fail[i] = ++j;
}
return fail;
}
vector<int> KMP(const string &s, const string &p){
int n = s.size(), m = p.size();
vector<int> fail = GetFail(p), ret;
for(int i=0, j=0; i<s.size(); i++){
while(j && s[i] != p[j]) j = fail[j-1];
if(s[i] == p[j]){
if(j + 1 == m) ret.push_back(i-m+1), j = fail[j];
else j++;
}
}
return ret;
}
```

6.3 가장 긴 팰린드롬 부분 문자열 - Manacher

Usage: $Manacher\backslash \Delta 8|\rfloor < \backslash X\backslash 0lX\lrcorner X$
Time Complexity: $O(N)$

```
// Time Complexity: O(N)
// 각문자를중심으로하는최장팰린드롬의반경을반환
// Manacher("abaaba") = {0,1,0,3,0,1,6,1,0,3,0,1,0}
// # a # b # a # a # b # a #
// 0 1 0 3 0 1 6 1 0 3 0 1 0
vector<int> Manacher(const string &inp){
int n = inp.size() * 2 + 1;
vector<int> ret(n);
string s = "#";
for(auto i : inp) s += i, s += "#";
for(int i=0, p=-1, r=-1; i<n; i++){
ret[i] = i <= r ? min(r-i, ret[2*p-i]) : 0;
while(i-ret[i]-1 >= 0 && i+ret[i]+1 < n && s[i-ret[i]-1] == s[i+ret[i]+1])
ret[i]++;
if(i+ret[i] > r) r = i+ret[i], p = i;
}
return ret;
}
```

6.4 문자열 매칭 - Z

Usage: $Z[i]\backslash S[0 : N]S[i : N]X\rfloor P8t$
Time Complexity: $O(N)$

```
// Time Complexity: O(N)
// Z[i] = LongestCommonPrefix(S[0:N], S[i:N])
// = S[0:N]과 S[i:N]이 앞에서부터 몇 글자 겹치는지
vector<int> Z(const string &s){
int n = s.size();
vector<int> z(n);
z[0] = n;
```

```
for(int i=1, l=0, r=0; i<n; i++){
if(i < r) z[i] = min(r-i-1, z[i-l]);
while(i+z[i] < n && s[i+z[i]] == s[z[i]]) z[i]++;
if(i+z[i] > r) r = i+z[i], l = i;
}
return z;
}
```

6.5 접미사 배열

Usage: $SuffixArray\backslash 0LCP0$
Time Complexity: $O(N \log N)$

```
// Time Complexity: O(N log N)
// LCP는1-based
pair<vector<int>, vector<int>> SuffixArray(const string &s){ // O(N log N)
int n = s.size(), m = max(n, 256);
vector<int> sa(n), lcp(n), pos(n), tmp(n), cnt(m);
auto counting_sort = [&](){
fill(cnt.begin(), cnt.end(), 0);
for(int i=0; i<n; i++) cnt[pos[i]]++;
partial_sum(cnt.begin(), cnt.end(), cnt.begin());
for(int i=n-1; i>=0; i--) sa[--cnt[pos[tmp[i]]]] = tmp[i];
};
for(int i=0; i<n; i++) sa[i] = i, pos[i] = s[i], tmp[i] = i;
counting_sort();
for(int k=1; ; k<<=1){
int p = 0;
for(int i=n-k; i<n; i++) tmp[p++] = i;
for(int i=0; i<n; i++) if(sa[i] >= k) tmp[p++] = sa[i] - k;
counting_sort();
tmp[sa[0]] = 0;
for(int i=1; i<n; i++){
tmp[sa[i]] = tmp[sa[i-1]];
if(sa[i-1]+k < n && sa[i]+k < n && pos[sa[i-1]] == pos[sa[i]] && pos[sa[i-1]+k] ==
pos[sa[i]+k]) continue;
tmp[sa[i]] += 1;
}
swap(pos, tmp); if(pos[sa.back()] + 1 == n) break;
}
for(int i=0, j=0; i<n; i++, j=max(j-1,0)){
if(pos[i] == 0) continue;
while(sa[pos[i]-1]+j < n && sa[pos[i]]+j < n && s[sa[pos[i]-1]+j] == s[sa[pos[i]]+j])
j++;
lcp[pos[i]] = j;
}
return {sa, lcp};
}
```

7 계산 기하

7.1 2차원 계산 기하 템플릿 + CCW

Usage: –
Time Complexity: –

```
#include <bits/stdc++.h>
#define x first
#define y second
using namespace std;
using ll = long long;
using Point = pair<ll, ll>;
Point operator + (Point p1, Point p2){ return {p1.x + p2.x, p1.y + p2.y}; }
Point operator - (Point p1, Point p2){ return {p1.x - p2.x, p1.y - p2.y}; }
ll operator * (Point p1, Point p2){ return p1.x * p2.x + p1.y * p2.y; } // 내적
ll operator / (Point p1, Point p2){ return p1.x * p2.y - p2.x * p1.y; } // 외적
int Sign(ll v){ return (v > 0) - (v < 0); } // 양수면+1, 음수면-1, 0이면0 반환
ll Dist(Point p1, Point p2){ return (p2 - p1) * (p2 - p1); } // 두점거리제곱
ll SignedArea(Point p1, Point p2, Point p3){ return (p2 - p1) / (p3 - p1); }
int CCW(Point p1, Point p2, Point p3){ return Sign(SignedArea(p1, p2, p3)); }
```

7.2 360도 각도 정렬

Usage: –
Time Complexity: –

```
/* y축
// ↑
// 2 1
// 4 -1 0 →x축
// 6 7
// 원점기준각도정렬
// x축위에있는점이가장먼저, 그리고반시계방향으로
*/
int QuadrantID(const Point p){
static int arr[9] = { 5, 4, 3, 6, -1, 2, 7, 0, 1 };
return arr[Sign(p.x)*3+Sign(p.y)+4];
}
sort(points.begin(), points.end(), [&](auto p1, auto p2){
if(QuadrantID(p1) != QuadrantID(p2)) return QuadrantID(p1) < QuadrantID(p2);
else return p1 / p2 > 0; // 반시계
});
```

7.3 다각형 넓이

Usage: –
Time Complexity: –

```
// 다각형의넓이의2배를반환, 항상정수, O(N)
ll PolygonArea(const vector<Point> &v){
ll res = 0;
for(int i=0; i<v.size(); i++) res += v[i] / v[(i+1)%v.size()];
return abs(res);
}
```

7.4 선분 교차 판정

Usage: –
Time Complexity: –

```
// 선분교차- 선분ab와선분cd가만나면true
bool Cross(Point s1, Point e1, Point s2, Point e2){
int ab = CCW(s1, e1, s2) * CCW(s1, e1, e2);
int cd = CCW(s2, e2, s1) * CCW(s2, e2, e1);
if(ab == 0 && cd == 0){
if(s1 > e1) swap(s1, e1);
if(s2 > e2) swap(s2, e2);
return !(e1 < s2 || e2 < s1);
}
return ab <= 0 && cd <= 0;
}
// 교차하지않으면0
// 교점이무한히많으면-1
// 교점이1개면1 반환하고res에교점저장
int Cross(Point s1, Point e1, Point s2, Point e2, pair<double, double> &res){
if(!Cross(s1, e1, s2, e2)) return 0;
ll det = (e1 - s1) / (e2 - s2);
if(!det){
if(s1 > e1) swap(s1, e1);
if(s2 > e2) swap(s2, e2);
if(e1 == s2){ res = s2; return 1; }
if(e2 == s1){ res = s1; return 1; }
```

```
return -1;
}
res.x = s1.x + (e1.x - s1.x) * ((s2 - s1) / (e2 - s2) * 1.0 / det);
res.y = s1.y + (e1.y - s1.y) * ((s2 - s1) / (e2 - s2) * 1.0 / det);
return 1;
}
```

7.5 다각형 내부 판별

Usage: $PointInPolygon < \backslash_{1}t\Delta\check{}_{1}\Xi_{1}$

Time Complexity: $O(N)$

```
// 다각형내부또는경계위에p가있으면true, O(N)
bool PointInPolygon(const vector<Point> &v, Point p){
int n = v.size(), cnt = 0;
Point p2(p.x+1, 1'000'000'000 + 1); // 좌표범위보다큰수
for(int i=0; i<n; i++){
int j = i + 1 < n ? i + 1 : 0;
if(min(v[i],v[j]) <= p && p <= max(v[i],v[j]) && CCW(v[i], v[j], p) == 0) return
true;
if(SegmentIntersection(v[i], v[j], p, p2)) cnt++;
}
return cnt % 2 == 1;
}
```

7.6 볼록 껍질 - Graham Scan

Usage: $ConvexHull\backslash_{1}DhX\Gamma@]\Delta\check{}$

Time Complexity: $O(N \log N)$

```
// 모든점을포함하는가장작은볼록다각형, O(N log N)
vector<Point> ConvexHull(vector<Point> points){
if(points.size() <= 1) return points;
swap(points[0], *min_element(points.begin(), points.end()));
sort(points.begin()+1, points.end(), [&](auto a, auto b){
int dir = CCW(points[0], a, b);
if(dir != 0) return dir > 0;
return Dist(points[0], a) < Dist(points[0], b);
});
vector<Point> hull;
for(auto p : points){
while(hull.size() >= 2 && CCW(hull[hull.size()-2], hull.back(), p) <= 0)
hull.pop_back();
hull.push_back(p);
}
return hull;
}
```

7.7 가장 먼 두 점 - Rotating Calipers

Usage: $\lrcorner) < \backslash\check{}, \emptyset]\Delta\check{}D\%\text{œ}D\Gamma < P_{1}$

Time Complexity: $O(N)$

```
// 가장먼두점을구하는함수, O(N)
// 주의: hull은반시계방향으로정렬된볼록다각형이어야함
pair<Point, Point> Calipers(vector<Point> hull){
int n = hull.size(); ll mx = 0; Point a, b;
for(int i=0, j=0; i<n; i++){
while(j + 1 < n && (hull[i+1] - hull[i]) / (hull[j+1] - hull[j]) >= 0){
ll now = Dist(hull[i], hull[j]);
if(now > mx) mx = now, a = hull[i], b = hull[j];
j++;
}
ll now = Dist(hull[i], hull[j]);
if(now > mx) mx = now, a = hull[i], b = hull[j];
}
return {a, b};
}
```

7.8 볼록 다각형 내부 판별

Usage: $\lrcorner) < \backslash\check{}, \emptyset]\Delta\check{}\emptyset_{1}t_{1}\Xi_{1}$

Time Complexity: $O(\log N)$

```
// 다각형내부또는경계위에p가있으면true, O(log N)
// 주의: v는반시계방향으로정렬된볼록다각형이어야함
bool PointInConvexPolygon(const vector<Point> &v, const Point &pt){
if(CCW(v[0], v[1], pt) < 0) return false; int l = 1, r = v.size() - 1;
while(l < r){
int m = l + r + 1 >> 1;
if(CCW(v[0], v[m], pt) >= 0) l = m; else r = m - 1;
}
if(l == v.size() - 1) return CCW(v[0], v.back(), pt) == 0 && v[0] <= pt && pt <=
v.back();
return CCW(v[0], v[l], pt) >= 0 && CCW(v[l], v[l+1], pt) >= 0 && CCW(v[l+1], v[0], pt)
>= 0;
}
```

8 기타

8.1 가장 긴 증가하는 부분 수열(LIS)

Usage: $LISh\lrcorner\backslash\Gamma 4\Gamma X\lrcorner\lrcorner X$

Time Complexity: $O(N \log N)$

```
// Time Complexity: O(N log N)
// LIS({10, 20, 10, 30, 20, 50}) = {10, 20, 30, 50}
vector<int> LIS(const vector<int> &v){
int n = v.size();
vector<int> pos(n), lis, res;
for(int i=0; i<n; i++){
if(lis.empty() || lis.back() < v[i]){
lis.push_back(v[i]); pos[i] = lis.size();
}
else{
int idx = lower_bound(lis.begin(), lis.end(), v[i]) - lis.begin();
lis[idx] = v[i]; pos[i] = idx + 1;
}
}
int len = lis.size();
for(int i=n-1; i>=0; i--) if(pos[i] == len) res.push_back(v[i]), len--;
reverse(res.begin(), res.end());
return res;
}
```

8.2 삼분 탐색

Usage: $Checkh\lrcorner X\backslash\grave{}D^{-}\Xi X$

Time Complexity: –

```
while(s + 3 <= e){ // get minimum / when multiple answer, find minimum `s`
T l = (s + s + e) / 3, r = (s + e + e) / 3;
if(Check(l) > Check(r)) s = l; else e = r;
}
T mn = INF, idx = s;
for (T i=s; i<=e; i++) if(T now = Check(i); now < mn) mn = now, idx = i;
```

8.3 C++ 랜덤, GCC 확장, 비트마스킹 트릭

Usage: –
Time Complexity: –

```
mt19937 rd((unsigned)chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution<int> rnd_int(l, r); // rnd_int(rd)
uniform_real_distribution<double> rnd_real(0, 1); // rnd_real(rd)
/////
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/rope>
using namespace __gnu_pbds; //ordered_set : find_by_order(order), order_of_key(key)
using namespace __gnu_cxx; //crope : append(str), substr(s, e), at(idx)
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
/////
int __builtin_clz(int x);// number of leading zero
int __builtin_ctz(int x);// number of trailing zero
int __builtin_popcount(int x);// number of 1-bits in x
lsb(n): (n & -n); // last bit (smallest)
floor(log2(n)): 31 - __builtin_clz(n | 1);
floor(log2(n)): 63 - __builtin_clzll(n | 1);
long long next_perm(long long v){
long long t = v | (v-1);
return (t + 1) | (((~t & -~t) - 1) >> (__builtin_ctz(v) + 1));
}
int frq(int n, int i) { // # of digit i in [1, n]
int j, r = 0;
for (j = 1; j <= n; j *= 10) if (n / j / 10 >= !i) r += (n / 10 / j - !i) * j + (n / j %
10 > i ? j : n / j % 10 == i ? n % j + 1 : 0);
return r;
}
```

8.4 빠른 입력(Fast Input from STDIN)

Usage: $STDIN < \backslash 0 `t\%$
Time Complexity: –

```
namespace io {
const signed IS=1<<20;
char I[IS+1],*J=I;
inline void daer(){if(J>=I+IS-64){
char*p=I;do*p++=*J++;
while(J!=I+IS);p[read(0,p,I+IS-p)]=0;J=I;}}
template<int N=10,typename T=int>inline T getu(){
daer();T x=0;int k=0;do x=x*10+*J-'0';
while(*++J>='0'&&++k<N);++J;return x;}
template<int N=10,typename T=int>inline T geti(){
daer();bool e=*J=='-';J+=e;return(e?-1:1)*getu<N,T>();}
struct f{f(){I[read(0,I,IS)]=0;}}flu;
};
```

9 Other Stuffs

9.1 Sum Over Subsets Dynamic Programming

Usage: –
Time Complexity: –

```
//iterative version
for(int mask = 0; mask < (1<<N); ++mask){
dp[mask][-1] = A[mask];
//handle base case separately (leaf states)
for(int i = 0;i < N; ++i){
if(mask & (1<<i))
dp[mask][i] = dp[mask][i-1] + dp[mask^(1<<i)][i-1];
// else
dp[mask][i] = dp[mask][i-1];
}
F[mask] = dp[mask][N-1];
}
//memory optimized, super easy to code.
for(int i = 0; i<(1<<N); ++i)
F[i] = A[i];
for(int i = 0;i < N; ++i) for(int mask = 0; mask < (1<<N); ++mask){
if(mask & (1<<i))
F[mask] += F[mask^(1<<i)];
}
```

9.2 Edmond's Algorithm (Directed-MST)

Usage: –
Time Complexity: $O(VE)$

```
// Time Complexity: O(V E)
// credit: https://gina65.tistory.com/23
struct EDGE {
int u, v;
ll c;
EDGE(int u, int v, ll c): u(u), v(v), c(c) {}
};
// set of all edges of a graph (vertices are zero-based)
vector<EDGE> ed;
// returns the weight of the directed minimum spanning tree rooted at vertex r (zero-based)
// N denotes the number of vertices
ll dmst(int N, int r) {
vector<int> mnu(N), lbl(N);
vector<ll> mnc(N);
ll ans = 0;
while(true) {
fill(mnc.begin(), mnc.end(), LLONG_MAX);
for(EDGE e : ed)
if(e.u != e.v && e.c < mnc[e.v]) {
mnc[e.v] = e.c;
mnu[e.v] = e.u;
}
mnc[r] = 0;
for(int i = 0; i < N; i++) ans += mnc[i];
fill(lbl.begin(), lbl.end(), -1);
int K = 0;
lbl[r] = K++;
for(int i = 0; i < N; i++) if(lbl[i] == -1) {
int x;
for(x = i; lbl[x] == -1; x = mnu[x]) lbl[x] = -2;
if(lbl[x] == -2) {
for(; lbl[x] == -2; x = mnu[x]) lbl[x] = K;
K++;
}
for(x = i; lbl[x] == -2; x = mnu[x]) lbl[x] = K++;
}
if(K == N) break;
N = K;
for(EDGE &e : ed) {
e.c -= mnc[e.v];
e.u = lbl[e.u];
e.v = lbl[e.v];
}
r = lbl[r];
}
```

```
return ans;
}
```

9.3 LCA - ETT + RMQ

Usage: ΞP˘ıXLCA¸

Time Complexity: *preprocessing* : $O(N \log N)$

```
// preprocessing: O(NlogN)
// query: O(1)
int N;
vector<int> adj[100001];
int enter[100001];
pair<int, int> st[18][200001];
void dfs(int cur, int prev) {
static int t;
enter[cur] = ++t;
st[0][t] = {enter[cur], cur};
for (int next : adj[cur]) {
if (next != prev) {
dfs(next, cur);
st[0][++t] = {enter[cur], cur};
}
}
}
void pre() {
for (int i = 1; (1<<i) <= 2*N; i++) {
for (int j = 1; j+(1<<i)-1 <= 2*N; j++) {
st[i][j] = min(st[i-1][j], st[i-1][j+(1<<(i-1))]);
}
}
}
int lca(int u, int v) {
if ((u = enter[u]) > (v = enter[v]))
swap(u, v);
int l = __lg(v-u+1);
return min(st[l][u], st[l][v-(1<<l)+1]).second;
}
```

9.4 LCA - Binary Lifting

Usage: –

Time Complexity: –

```
// preprocessing: O(NlogN)
// query: O(logN)
int N, H;
vector<int> adj[100001];
int dep[100001] = {-1};
int st[18][100001];
void dfs(int cur, int prev, int h) {
H = max(H, __lg(h+1));
dep[cur] = h;
st[0][cur] = prev;
for (int i = 1; (1<<i)<=h; i++)
st[i][cur] = st[i-1][st[i-1][cur]];
for (int next : adj[cur]) {
if (next != prev)
dfs(next, cur, h+1);
}
}
int lca(int u, int v) {
if (dep[u] < dep[v])
swap(u, v);
for (int i = H; i >= 0; i--) {
if (dep[st[i][u]] >= dep[v])
u = st[i][u];
}
for (int i = H; i >= 0; i--) {
if (st[i][u] != st[i][v]) {
u = st[0][st[i][u]];
v = st[0][st[i][v]];
}
}
return u == v ? u : st[0][u];
}
```

9.5 Eulerian Path/Circuit

Usage: –

Time Complexity: –

```
// ans contains the final euler circult
// In case of an Eulerian trail, add a temporary edge between two odd vertices,
// and remove it later.
vector<set<int>> graph;
vector<int> ans;
void dfs(int v) {
while (!graph[v].empty()) {
int u = *graph[v].begin();
graph[u].erase(v);
graph[v].erase(u);
dfs(u);
}
ans.push_back(v);
}
// • Eulerian cycle (undirected): all vertices must have even degree.
// • Eulerian cycle (directed): every vertex has equal indegree and outdegree.
// • Eulerian path (undirected): exactly zero or two vertices must have odd degree.
// • Eulerian path (directed): at most one vertex has outin = 1 and at most one vertex has inout
= 1.
// all other vertices must have equal indegree and outdegree.
```

9.6 Maximum Subarray Segment Tree

Usage: –

Time Complexity: –

```
// segment tree for finding the maximum continuous sum in [l, r].
// struct seg
{
// left, right, max, total.
ll l, r, m, t;
};
seg seg_merge(seg a, seg b)
{
seg s;
s.l = max(a.l, a.t + b.l);
s.r = max(b.r, b.t + a.r);
s.m = max(max(a.m, b.m), a.r + b.l);
s.t = a.t + b.t;
return s;
}
int A[100001];
seg segtree[400001];
void build(int cur, int begin, int end)
{
if (begin == end)
{
segtree[cur] = (seg) {A[begin], A[begin], A[begin], A[begin]};
```

```
return;
}
int mid = (begin + end)/2;
build(cur*2, begin, mid);
build(cur*2+1, mid+1, end);
segtree[cur] = seg_merge(segtree[cur*2], segtree[cur*2+1]);
}
seg query(int cur, int begin, int end, int qbegin, int qend)
{
if (qend < begin || end < qbegin)
return (seg) {INT_MIN, INT_MIN, INT_MIN, 0};
if (qbegin <= begin && end <= qend)
return segtree[cur];
int mid = (begin + end)/2;
seg l = query(cur*2, begin, mid, qbegin, qend);
seg r = query(cur*2+1, mid+1, end, qbegin, qend);
return seg_merge(l, r);
}
```

9.7 Offline Queries - Mo's Algorithm

Usage: –
Time Complexity: $O(Q \log Q + (N + Q)N)$

```
// Time Complexity: O(Q log Q + (N + Q)
// √
N)
// program to find the number of distinct elements in [l, r].
// struct query
{
static int block_sz;
int i, j, idx;
bool operator<(const query& a) const
{
return make_pair(i/block_sz, j) < make_pair(a.i/block_sz, a.j);
}
};
// this should be set to sqrt(N) upon reading the input.
int query::block_sz = 0;
int N, M;
int A[100001];
query Q[100000];
int R[100000]; // result vector.
void collect()
{
static int freq[1000001];
int cnt = 0;
auto insert = [&](int idx)
{
if (!freq[A[idx]]++)
cnt++;
};
auto remove = [&](int idx)
{
if (!--freq[A[idx]])
cnt--;
};
sort(Q, Q+M);
int i = 1;
int j = 0;
for (int k = 0; k < M; k++)
{
query q = Q[k];
while (j < q.j)
insert(++j);
while (j > q.j)
remove(j--);
while (i > q.i)
insert(--i);
while (i < q.i)
remove(i++);
R[q.idx] = cnt;
}
}
```

9.8 2-Dimensional Segment Tree - Recursive

Usage: –
Time Complexity: –

```
#define lseg(x) (x*2)
#define rseg(x) (x*2 + 1)
#define mid(x, y) ((x + y)/2)
int N;
int grid[1025][1025];
int seg[4097][4097];
void build_y(int cur_x, int begin_x, int end_x, int cur_y, int begin_y, int end_y)
{
int mid_y = mid(begin_y, end_y);
if (begin_y == end_y)
{
if (begin_x == end_x)
seg[cur_x][cur_y] = grid[begin_x][begin_y];
// else
seg[cur_x][cur_y] = seg[lseg(cur_x)][cur_y] + seg[rseg(cur_x)][cur_y];
}
// else
{
build_y(cur_x, begin_x, end_x, lseg(cur_y), begin_y, mid_y);
build_y(cur_x, begin_x, end_x, rseg(cur_y), mid_y+1, end_y);
seg[cur_x][cur_y] = seg[cur_x][lseg(cur_y)] + seg[cur_x][rseg(cur_y)];
}
}
void build_x(int cur_x, int begin_x, int end_x)
{
int mid_x = mid(begin_x, end_x);
if (begin_x != end_x)
{
build_x(lseg(cur_x), begin_x, mid_x);
build_x(rseg(cur_x), mid_x+1, end_x);
}
build_y(cur_x, begin_x, end_x, 1, 1, N);
}
void update_y(int cur_x, int begin_x, int end_x, int cur_y, int begin_y, int end_y, int x,
int y, int val)
{
if (begin_y == end_y)
{
if (begin_x == end_x)
seg[cur_x][cur_y] = val;
// else
seg[cur_x][cur_y] = seg[lseg(cur_x)][cur_y] + seg[rseg(cur_x)][cur_y];
return;
}
int mid_y = mid(begin_y, end_y);
if (y <= mid_y)
update_y(cur_x, begin_x, end_x, lseg(cur_y), begin_y, mid_y, x, y, val);
// else
```

```
update_y(cur_x, begin_x, end_x, rseg(cur_y), mid_y+1, end_y, x, y, val);
seg[cur_x][cur_y] = seg[cur_x][lseg(cur_y)] + seg[cur_x][rseg(cur_y)];
}
void update_x(int cur_x, int begin_x, int end_x, int x, int y, int val)
{
int mid_x = mid(begin_x, end_x);
if (begin_x != end_x)
{
if (x <= mid_x)
update_x(lseg(cur_x), begin_x, mid_x, x, y, val);
// else
update_x(rseg(cur_x), mid_x+1, end_x, x, y, val);
}
update_y(cur_x, begin_x, end_x, 1, 1, N, x, y, val);
}
int query_y(int cur_x, int cur_y, int begin_y, int end_y, int qbegin_y, int qend_y)
{
if (qend_y < begin_y || end_y < qbegin_y)
return 0;
if (qbegin_y <= begin_y && end_y <= qend_y)
return seg[cur_x][cur_y];
int mid_y = mid(begin_y, end_y);
int l = query_y(cur_x, lseg(cur_y), begin_y, mid_y, qbegin_y, qend_y);
int r = query_y(cur_x, rseg(cur_y), mid_y+1, end_y, qbegin_y, qend_y);
return l + r;
}
int query_x(int cur_x, int begin_x, int end_x, int qbegin_x, int qend_x, int qbegin_y, int
qend_y)
{
if (qend_x < begin_x || end_x < qbegin_x)
return 0;
if (qbegin_x <= begin_x && end_x <= qend_x)
return query_y(cur_x, 1, 1, N, qbegin_y, qend_y);
int mid_x = mid(begin_x, end_x);
int l = query_x(lseg(cur_x), begin_x, mid_x, qbegin_x, qend_x, qbegin_y, qend_y);
int r = query_x(rseg(cur_x), mid_x+1, end_x, qbegin_x, qend_x, qbegin_y, qend_y);
return l + r;
}
```

9.9 2-Dimensional Segment Tree - Iterative

Usage: –
Time Complexity: –

```
// credit: https://infossm.github.io/blog/2019/11/15/2D-segment-tree/
// all indices here are zero-based.
class Seg2D{
// public:
int n, m;
vector<vector<int>> a;
Seg2D(int n, int m) : n(n), m(m), a(2*n,vector<int>(2*m)){}
void init(vector<vector<int>>& val){
for(int i = 0; i < n; i++){
for(int j = 0; j < m; j++)
a[i+n][j+m] = val[i][j];
}
for(int i = n; i < 2*n; i++){
for(int j = m-1; j > 0; j--) a[i][j] = a[i][j<<1]+a[i][j<<1|1];
}
for(int i = n-1; i > 0; i--){
for(int j = 1; j < 2*m; j++){
a[i][j] = a[i<<1][j] + a[i<<1|1][j];
}
}
void upd(int x, int y, int val) {
a[x+n][y+m] = val;
for(int i = y+m; i > 1; i >>= 1)
a[x+n][i>>1] = a[x+n][i] + a[x+n][i^1];
for(x = x+n; x > 1; x >>= 1){
for(int i = y+m; i >= 1; i >>= 1){
a[x>>1][i] = a[x][i]+a[x^1][i];
}
}
}
int query1D(int x, int y1, int y2){
int ret = 0;
for(y1 += m, y2 += m+1; y1 < y2; y1 >>= 1, y2 >>= 1){
if(y1 & 1) ret += a[x][y1++];
if(y2 & 1) ret += a[x][--y2];
}
return ret;
}
int query(int x1, int y1, int x2, int y2) {
int ret = 0;
for(x1 += n, x2 += n+1; x1 < x2; x1 >>= 1, x2 >>= 1){
if(x1&1) ret += query1D(x1++, y1, y2);
if(x2&1) ret += query1D(--x2, y1, y2);
}
return ret;
}
};
```

9.10 2-Dimensional Fenwick Tree

Usage: –
Time Complexity: –

```
// credit: https://github.com/ncduy0303/Competitive-Programming/
// all indices are one-based.
#define MAX_N 5005
#define MAX_M 5005
int n, m, q;
ll ft[MAX_N][MAX_M], grid[MAX_N][MAX_M];
void update(int x, int y, int v) {
for (int tx = x; tx <= n; tx += tx&-tx)
for (int ty = y; ty <= m; ty += ty&-ty)
ft[tx][ty] += v;
}
ll sum(int x, int y) {
ll res = 0;
for (int tx = x; tx; tx -= tx&-tx)
for (int ty = y; ty; ty -= ty&-ty)
res += ft[tx][ty];
return res;
}
ll query(int x1, int y1, int x2, int y2) {
return sum(x2, y2) - sum(x1 - 1, y2) - sum(x2, y1 - 1) + sum(x1 - 1, y1 - 1);
}
```

10 template

10.1 cpp template

Usage: –
Time Complexity: –

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
#define fastIO ios_base::sync_with_stdio(false); cin.tie(NULL)
#define endl "\n"
#define INF 1000000001
#define ALL(v) v.begin(), v.end()
#define x first
#define y second
#define pb push_back

using ll = long long;
using pii = pair<int, int>;
using pll = pair<ll, ll>;

template <typename T>
using min_heap = priority_queue<T, vector<T>, greater<T>>;

template <typename T>
using max_heap = priority_queue<T>;

void solve(){

}

int main(){
    fastIO;

    int T = 1;
    while(T--){
        solve();
    }

    return 0;
}
```

10.2 Makefile

```
CXX := g++
CPPFLAGS := -DONLINE_JUDGE -DDOMJUDGE
CXXFLAGS := -g -O2 -std=gnu++20 -mno-avx -Wall -pipe
LDFLAGS := -static

ifeq ($(OS),Windows_NT)
    EXEEXT := .exe
else
    EXEEXT :=
endif

.PHONY: all
all: main$(EXEEXT)

%$(EXEEXT): %.cpp Makefile
    $(CXX) $(CPPFLAGS) $(CXXFLAGS) $< $(LDFLAGS) -o $@
```

11 문제 풀이 체크리스트

- 단순한 방법에서 시작할 수 있을까? (Brute Force)
- 내가 문제를 푸는 과정을 수식화할 수 있을까? (예제를 직접 해결해보면서)
- 문제를 단순화할 수 없을까?
- 그림으로 그려볼 수 있을까?
- 수식으로 표현할 수 있을까?
- 문제를 분해할 수 있을까?
- 뒤에서부터 생각해서 풀 수 있을까?
- 순서를 강제할 수 있을까?
- 특정 형태의 답만을 고려할 수 있을까? (정규화)
- **구간을 통째로 가져간다:** 플로우 + 적당한 자료구조 $(i, i+1, k, 0)$, $(s, e, 1, w)$, $(N, T, k, 0)$
- $a = b$**:** a만 움직이기, b만 움직이기, 두 개 동시에 움직이기, 반대로 움직이기
- 말도 안 되는 것들을 한 번은 생각해보기 / "당연하다고 생각한 것" 다시 생각해보기
- **확률:** DP, 이분 탐색 (NYPC 2019 Finals C)
- **최대/최소:** 이분 탐색, 그리디 (Prefix 고정, Exchange Argument), DP (순서 고정)
- 한 가지 풀이에 지나치게 시간을 많이 쓰지 말자. 다른 문제도 한번 읽어보고, 차분히 숨을 돌리며 다른 풀이도 생각해보자.
- 접근조차 되지 않는 문제는 어떤 제한을 제거했을 때 문제가 쉬워지는지 판단해볼 필요가 있다.
- 억지로 문제를 뚫는 것보다 적절한 관찰을 통하여 좋은 풀이를 작성해내는 것이 더 효과적일 수 있다.
- Sometimes, a seemingly naive $O(N^2)$ might turn out to be $O(N \log N)$, or even $O(N)$.

12 empty











